PHINS — Platform Data Architecture

PHINS Platform Data Architecture 2.0

Objective

PHINS is evolving from a demo-style, mixed in-memory platform into an insurance, health, savings, banking, investment, and actuarial operating core with durable lineage. The target architecture below establishes one principle:

> Every critical business event must be auditable, append-only, tamper-evident, and reconcilable across API, workflow, BI, and actuarial views.

Target State

Core controls

- Operational events are persisted to `audit_logs`
- Financial and non-financial lineage is persisted to `platform_ledger_entries`
- Runtime transaction state remains backward compatible through `TRANSACTION_LEDGER`
- Ledger entries carry:
- `sequence_no`
- `previous_hash`
- `entry_hash`
- entity and customer references
- canonical payload for replay and BI lineage
- Pipeline integrity dashboards validate ledger chain health before trusting totals

Component UML

```
classDiagram
    class PortalHandler {
      +record_transaction()
      +GET /api/audit
      +GET /api/diagnostics/ledger-integrity
    }

    class AuditService {
      +log(actor, action, entity, entity_id, details)
      +recent(limit)
    }

    class PlatformEventLedgerService {
      +append_event(...)
      +ensure_hash_chain()
      +get_integrity_summary()
    }

    class PipelineIntegrityService {
      +validate_policy_pipeline(policy_id)
      +get_bi_dashboard_data()
    }
    class DatabaseManager {
```

```
  +audit
  +platform_ledger
}

class AuditLog {
  +id
  +timestamp
  +action
  +entity_type
  +entity_id
}

class PlatformLedgerEntry {
  +id
  +sequence_no
  +ledger_type
  +event_type
  +previous_hash
  +entry_hash
}

PortalHandler --> AuditService : logs actions
PortalHandler --> PlatformEventLedgerService : appends transactions/events
AuditService --> DatabaseManager : persists audit trail
PlatformEventLedgerService --> DatabaseManager : persists ledger chain
DatabaseManager --> AuditLog
DatabaseManager --> PlatformLedgerEntry
PipelineIntegrityService --> PlatformEventLedgerService : reconciles lineage
```

Sequence UML

```
sequenceDiagram
    participant API as API Request
    participant Portal as PortalHandler
    participant Audit as AuditService
    participant Ledger as PlatformEventLedgerService
    participant DB as SQL Database
    participant BI as BI / Integrity Services

    API->>Portal: create/update/pay/approve action
    Portal->>Audit: log(actor, action, entity, entity_id, details)
    Audit->>DB: insert audit_logs row
    Audit->>DB: insert platform_ledger_entries row (audit scope)

    Portal->>Ledger: append_event(transaction/event payload)
    Ledger->>Ledger: compute sequence_no + previous_hash + entry_hash
    Ledger->>DB: insert platform_ledger_entries row
    Ledger-->>Portal: normalized append-only entry

    BI->>Ledger: reconcile ledger chain
    Ledger-->>BI: integrity summary + latest hash + anomalies
```

Integrity contract

1. `audit_logs` capture actor/action/entity semantics.
2. `platform_ledger_entries` capture append-only lineage for all strategic events.
3. `TRANSACTION_LEDGER` remains API-compatible but is no longer a raw mutable sink.
4. BI and actuarial summaries must treat invalid ledger chains as degraded data.
5. Legacy snapshot data is repaired through hash-chain backfill on load.

Recommended next increments

1. Add repository-backed persistence for health wallets, investments, and community data.

2. Replay legacy JSON snapshot state into `platform_ledger_entries` as a one-time migration.
3. Add nightly ledger reconciliation jobs and alerting.
4. Expand BI dashboards to show ledger coverage by domain: insurance, health, banking, investments, community.